

Reflection

Developing and documenting part B of my major work, Candlelight, over the last few months has been a massive learning experience for me as both a student and a budding developer. From learning how to implement complex algorithms such as A* and Cellular Automata, to resisting the looming temptations of scope creep, to cutting and redesigning planned features and to doing my best in balancing documentation and practical work, I can confidently say that I have matured as a developer - though there are still things to work on which require polish and time. Beginning with project successes and things that went better than expected, I can think of no better example than my implementation of A* pathfinding. In both my initial development and part 1 design document, my Enemy class was designed to operate with a fairly simple, follow-the-player style movement, following a straight generated by a simple Vector2 velocity. Candlelight consists of many tight spaces, complex passageways and edged corners, meaning that the enemy would often get stuck on walls and obstacles, never providing a convincing challenge to the far more technically able player. After implementing A*, something I was nervous about implementing given its complexity was above other algorithms I had previously developed on my own, the enemy was successfully able to navigate through any and all obstacles, providing a far more entertaining and functional player experience - not something I had expected to be able to implement, so this was a major success. Towards the latter end of Candlelight's development process, I implemented a simple F3 debug overlay. This also proved to be a success given it provided empirical evidence into some of the issues limiting performance of my game, such as allowing me to view fundamental in-game metrics such as FPS and total entities. Another thing that went well was my implementation of Cellular Automata. This algorithm was another I was unsure of implementing, and had planned to design 5-10 handmade dungeon rooms which I could stitch together to form many unique layouts, however I believe this could become very stale for the user after several play attempts, especially given Candlelight is specifically designed to be replayed many times as the user attempts to beat their previous highscore, and cellular automata is a way to work around that with continuous generation of new and unique dungeon rooms. Originally, I had my room layouts defined in *dungeon.py*, which I built a custom compiler for, reading huge grid lists of [0, 1, 1, 0, 1] to define which tiles are floors, walls or doors. After implementing my new generation algorithm, the game was noticeably more enjoyable and had more of a flow to the gameplay, so I would label organic cave generation as a definite success. In terms of things that went less than well, I initially was too undisciplined in my version control - I almost lost an entire iteration of my code until I realised I had a copy I'd put on google drive to transfer between devices. From then, every session I spent coding ended with a commit to GitHub, sometimes with 2 commits in one day if I was working on major features. There was also one particular issue perpetrated by my use of Try/Except. I was working on player collision with solid objects, and had an except loop passing AttributeErrors. It looked like everything was fine because no error was being thrown and as a result I couldn't find the base of the issue - after spending too much time looking through my code I realised my Candle object had an attribute under a different name than I had thought, obviously throwing an AttributeError, which had just been breezed past by the try/except. After this incident I removed all my try/excepts from my code bar one and pivoted to a cleaner coding methodology - an example of defensive coding gone wrong.

Changes I'd make if I was given a second chance would be firstly maintaining a consistent, professional development methodology. Some structures and patterns of my code changed massively during my development cycle - for example the previously mentioned misuse of try/except. More drastically, my code originally became bloated in the game and menu loops, with outstanding and duplicated logic where I should have simply used a simpler subroutine or split tasks between multiple functions. Towards the end of the development of Candlelight, I implemented a complete modular refactor, splitting tasks among simpler functions and ensuring my main loop was as readable and simplistic as possible, consisting primarily of calling functions. If I had simply focused on developing and maintaining a consistent philosophy from the genesis of development, I could have spent more time working on either the documentation side of my project or refining existing features. Secondly, I had a slight issue in portioning my time more realistically across the theory/documentation and practical side of this project. The better half of the documentation side of things was left until the final week before submission, which admittedly was hard to manage given trials in the weeks before, but nonetheless I could have compiled things like my testing along my development cycle instead of leaving them individually in the code and having to gather and recount all of them long after I had implemented them. However, I was pleased with my version documentation in the comments of my Github commits, though I do admit they are partially incomplete because they miss some of the very early versions of my code which I never backed up. Lastly, maintaining a strict version control discipline would have saved me from a few hours of recovering lost code and allowed me to document the earliest versions of my code effectively. After my initial mistake, I did begin to upload my code to GitHub fairly consistently and after pretty much every coding session, though I do wish I began slightly earlier. My first GitHub commit was pushed on May 9, and I was fairly consistent at uploading throughout July and August which I am happy about. In conclusion, developing Candlelight has proven to be a learning experience for myself as a developer, as I have pushed myself beyond personal expectations to implement complex algorithms and introduce solutions to unique problems, learning the true value of 'good' coding practice (which I realise I have more than undervalued and ignored in my years as a Software student) and lessons on how and why to manage time between the more enjoyable, practical side of things which comes more naturally to me and documenting and justifying my development practices.